

CityTech TTP Summer Bootcamp

API and Insomnia

2026-06-15

Agenda

1. Review
2. Follow-Ups
3. APIs and REST
4. HTTP Methods and Status Codes
5. Manual Calls: Insomnia and Swagger
6. Calling APIs from Node.js
7. API Tokens and Security
8. Explore APIs

Review

What are the most important things you remember from last week?

Follow Up: Project Management

Keeping a project organized, on track, and unblocked.

- **What it is** — planning the work, tracking tasks, removing blockers, keeping the team aligned
- **Who does it** varies:
 - sometimes a **senior engineer** already on the team
 - sometimes a **dedicated PM** — generally doesn't code, only manages
 - sometimes **one of your peers**

It's a real career path — e.g. the **Google Project Management Certificate** is a common entry point.

Follow Up: PM Tools

Teams track their work in shared tools:

- **GitHub** (Issues, Projects) and **Jira** — issue / ticket tracking
- **Kanban boards** — columns like *To Do* → *In Progress* → *Done*
- Others you'll meet: Linear, Trello, Asana, Azure Boards

A real one to explore — Kubernetes' SIG project boards:

<https://github.com/orgs/kubernetes/projects>

Follow Up: AI Disclosure

Where your data goes can be a **legal** issue, not just a courtesy.

- Some companies restrict **where data may be sent or stored**
- e.g. using an AI agent on company code, when that AI's data centers are in **China**, could violate those restrictions
- For a **government agency (e.g. NASA)** or **patient health records (HIPAA)**, this can be a **serious** violation
- Always **disclose** your AI use — and check what's allowed first

AI Disclosure — Example

A simple disclosure you might put in your project's `README` :

AI usage: I used Claude to scaffold the initial component structure and to help debug a date-formatting bug. I read and understood all generated code, and no private or company data was shared.

Name the **tool**, **where** you used it (planning / building / debugging), and **what** data (if any) you shared.

What Is an API?

API = **A**pplication **P**rogramming **I**nterface — a way for programs to talk to each other.

- A **web API** lets your app request data (or actions) from a server
- It's **lightweight**: it exchanges plain **data** (usually **JSON**), not whole web pages or UI
- Examples: a weather app fetching the forecast, "Log in with Google", Stripe payments, embedding Google Maps

REST APIs

Most web APIs follow **REST** (Representational State Transfer) — a common style for HTTP APIs.

- **Resources** live at **URLs** (e.g. `/courses` , `/courses/42`)
- You act on them with standard **HTTP methods**
- **Stateless** — each request stands on its own

Why REST?

- **Simple** — just HTTP; nothing special to learn
- **Predictable** — resource URLs + standard verbs feel familiar across APIs
- **Cacheable** — GET responses can be cached → faster, less server load
- **Scalable** — because it's stateless, any server can handle any request
- **Language-agnostic** — any client or server can speak it
- **Ubiquitous** — huge ecosystem and tooling (Insomnia, Postman, libraries)

Stateless vs. Stateful

- **Stateless** (how REST works) — each request carries **everything** the server needs (e.g. your auth token); the server **remembers nothing** between requests
 - any server can handle any request → easy to scale
- **Stateful** — the server **remembers context** between requests (a server-side **session**: who you are, where you are)
 - needs sticky or shared sessions → harder to scale

Analogy: **stateless** = a counter where you repeat your full order every time; **stateful** = a waiter who remembers your table and running tab.

HTTP Methods

Every request includes an **HTTP method** (alongside the URL) that says what you want to do. The common ones map neatly to **CRUD**:

- **GET** — read data (Read)
- **POST** — create something new (Create)
- **PUT / PATCH** — update (Update)
- **DELETE** — remove (Delete)

The server's code decides what each method actually does for a given URL.

Examples: RESTful API Calls

The **method** + **URL** together describe the action:

GET	/courses	→ list all courses
GET	/courses/42	→ get course 42
POST	/courses	→ create one (JSON body)
PATCH	/courses/42	→ update part of course 42 (JSON body)
DELETE	/courses/42	→ delete course 42

POST and **PATCH** send data in the request body — that data is called the **payload**.

A Full POST Request

```
POST /courses HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer <token>

{
  "name": "Intro to CS",
  "day": "Tuesday",
  "start": "14:00",
  "end": "15:00"
}
```

Status Codes

Every response comes with a 3-digit status:

- **2xx** success · **3xx** redirect · **4xx** your mistake · **5xx** the server's mistake

Common ones:

- **401** Unauthorized — not logged in / no valid auth
- **404** Not Found — that resource doesn't exist
- **501** Not Implemented — the server doesn't support that
- **503** Service Unavailable — the server is down or overloaded

Full reference: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

Manual API Calls with Insomnia

Insomnia lets you send API requests by hand and inspect the responses.

- Pick a **method** (GET / POST / ...), enter a **URL**, add headers or a JSON body, then **Send**
- See the **status code**, the response **JSON**, and timing
- Try a **correct** REST call, then a **malformed** one (wrong method, bad URL, missing auth, broken JSON) and compare the results

Practice API: <https://catfact.ninja>

Swagger / OpenAPI

Swagger (built on the **OpenAPI** spec) is a standard way to **describe** a REST API — its endpoints, methods, parameters, and responses.

- **Swagger UI** turns that description into **interactive docs** in the browser
- Read each endpoint and even "**Try it out**" to make live calls — no separate client needed
- Example: catfact.ninja's homepage *is* a Swagger UI — <https://catfact.ninja/#/>

Introducing Node.js

Remember "**JavaScript everywhere**" from last week? **Node.js** is the runtime that runs JavaScript **outside the browser** — on your machine and on servers.

- The same language (JS / TS) you used for React, now for **scripts and backends**
- Comes with **yarn / npm** for packages
- Lets us **call APIs**, build servers, and run tools from the command line

Insomnia vs. Code for API Calls

- **Insomnia** — best for **quick, manual, exploratory** testing: poke an endpoint and eyeball the response
- `curl` — a **command-line** HTTP client: fast one-off calls, and easy to drop into shell scripts
- **Node.js (code)** — best for **repeatable, automated** calls: run the same requests every time, in a script or inside your app
- Rule of thumb: explore by hand (Insomnia or `curl`) → once it works, move it into **code** so it's repeatable

A New Node.js Project

```
mkdir api-demo && cd api-demo  
yarn init -y          # creates package.json  
yarn add <package>    # add any dependency you need
```

- `yarn init` sets up `package.json` — your project and its dependencies
- `yarn add <pkg>` installs a package into `node_modules`
- For API calls you won't need one — `fetch` is built in (next slide)

Calling an API with `fetch`

`fetch` is built into Node **18+** (and every modern browser) — no package to install.

```
const res = await fetch("https://catfact.ninja/fact");  
const data = await res.json();  
console.log(data.fact);
```

- `fetch(url)` returns a **Promise** → use `await`
- `res.json()` parses the JSON body (also a Promise)

Full docs (POST bodies, headers, and more): https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Synchronous vs. Asynchronous

- **Synchronous** — one line at a time; each waits for the one before it to finish
- **Asynchronous** — slow work (like a network / API call) runs **in the background**; your code keeps going and handles the result when it's ready
- API calls are **async** — you don't freeze the whole app waiting for a response

async / **await** lets you write async code that still reads top-to-bottom:

```
async function getFact() {  
  const res = await fetch("https://catfact.ninja/fact"); // waits here  
  const data = await res.json();                       // ...and here  
  console.log(data.fact);  
}
```

- **await** pauses inside the **async** function until the Promise resolves — without

Mini Code Along: Node.js API Calls

Build a small Node script that calls a real API — **catfact.ninja**.

Why APIs Need Tokens

Many APIs require an **API token** (a key) with each request. Why?

- **Paid / tiered access** — track who's on the free vs. paid plan
- **Revocable** — if a token leaks or is misused, revoke just that token (not whole account)
- **Authorized origins** — lock a key to specific URLs / domains. Some keys are *meant* to be public (e.g. Google Maps keys, committed in client code) — origin limits keep others from abusing them
- **Rate limiting** — cap how many requests you can make in a window
- **Auditing** — see who called what, and when

Keep **secret** tokens out of your repo (use env vars); public client-side keys (like Google Maps) are protected by **origin restrictions** instead.

Code Together: NASA API

Let's call NASA's open APIs together — grab a free key and try one (e.g. APOD, the Astronomy Picture of the Day).

<https://api.nasa.gov/>

Explore These APIs

- **NASA** — <https://api.nasa.gov/>
- **OpenWeatherMap** — <https://openweathermap.org/api>
- **Deck of Cards** — <https://deckofcardsapi.com/>
- **SpaceTraders** — <https://spacetraders.io/> ★ — a full RPG you play entirely through API calls